32.3 沙箱机制

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~280k tokens，输出 ~9k tokens（本节）

当 AI Agent 被赋予执行命令、读写文件等"工具能力"后，一个核心问题浮出水面：如果 Agent 被提示词注入攻击劫持，它能造成多大的损害？OpenClaw 的沙箱系统通过 Docker 容器隔离来回答这个问题——将非主会话的工具执行限制在受控的容器环境中，即使 Agent 行为异常也无法触及宿主机。

32.3.1 沙箱设计：非主会话的 Docker 隔离（`src/agents/sandbox.ts`）

沙箱模式

沙箱行为通过 `agents.defaults.sandbox.mode` 配置，有三种模式：

模式	行为
`"off"`	默认值。不使用沙箱，所有命令在宿主机执行
`"non-main"`	非主会话使用沙箱。主会话（Bot 所有者的 CLI 交互）不受限
`"all"`	所有会话都在沙箱中执行，包括主会话

`"non-main"` 是推荐的生产配置——它让 Bot 所有者保留完整的宿主机访问权限，同时将来自渠道（WhatsApp、Telegram 等）的会话隔离到容器中。

沙箱作用域

`scope` 决定了容器的生命周期粒度：

```
// src/agents/sandbox/types.ts

export type SandboxScope = "session" | "agent" | "shared";
```

作用域	容器粒度	适用场景
`"session"`	每个会话一个容器	最强隔离，不同会话完全独立
`"agent"`	每个 Agent 一个容器（默认）	同一 Agent 的多个会话共享容器
`"shared"`	所有 Agent 共享一个容器	最节约资源

32.3.2 沙箱配置解析（ `src/agents/sandbox/` ）

双层配置合并

沙箱配置支持全局默认值和 Agent 级别覆盖的双层结构。 `resolveSandboxConfigForAgent` 函数合并两层配置：

```
// src/agents/sandbox/config.ts（简化）

export function resolveSandboxConfigForAgent(cfg?: OpenClawConfig, agentId?: string): S
  const globalSandbox = cfg?.agents?.defaults?.sandbox;    // 全局默认
  const agentSandbox = resolveAgentConfig(cfg, agentId)?.sandbox;  // Agent 级别

  const scope = resolveSandboxScope({
    scope: agentSandbox?.scope ?? globalSandbox?.scope,
  });

  return {
    mode: agentSandbox?.mode ?? globalSandbox?.mode ?? "off",
    scope,
    workspaceAccess: agentSandbox?.workspaceAccess ?? globalSandbox?.workspaceAccess ??
    workspaceRoot: agentSandbox?.workspaceRoot ?? globalSandbox?.workspaceRoot ?? DEFAU
    docker: resolveSandboxDockerConfig({ scope, globalDocker: ..., agentDocker: ... }),
    browser: resolveSandboxBrowserConfig({ scope, globalBrowser: ..., agentBrowser: ...
    tools: resolveSandboxToolPolicyForAgent(cfg, agentId),
    prune: resolveSandboxPruneConfig({ scope, globalPrune: ..., agentPrune: ... }),
  };
}
```

Docker 配置的安全默认值

`resolveSandboxDockerConfig` 应用了一系列安全加固默认值：

```
// src/agents/sandbox/config.ts（简化）

export function resolveSandboxDockerConfig(params): SandboxDockerConfig {
  return {
    image: agentDocker?.image ?? globalDocker?.image ?? "openclaw-sandbox:bookworm-slim
    readOnlyRoot: agentDocker?.readOnlyRoot ?? globalDocker?.readOnlyRoot ?? true,
    tmpfs: agentDocker?.tmpfs ?? globalDocker?.tmpfs ?? ["/tmp", "/var/tmp", "/run"],
    network: agentDocker?.network ?? globalDocker?.network ?? "none",    // 无网络！
    capDrop: agentDocker?.capDrop ?? globalDocker?.capDrop ?? ["ALL"],   // 丢弃所有能力！
    env: { LANG: "C.UTF-8", ...globalDocker?.env, ...agentDocker?.env },
    // ... 资源限制（pids, memory, cpu, ulimits）
  };
}
```

默认值	安全含义
`readOnlyRoot: true`	根文件系统只读，防止修改系统文件
`tmpfs: ["/tmp", "/var/tmp", "/run"]`	临时目录使用内存文件系统，容器销毁即清除
`network: "none"`	完全禁用网络访问，防止数据外泄
`capDrop: ["ALL"]`	丢弃所有 Linux 能力（capabilities），最小权限

> **衍生解释**：**Linux Capabilities** 是一种细粒度的特权划分机制，将传统的 root 超级用户权限拆分为 40+ 个独立的能力（如 `CAP_NET_ADMIN` 管理网络、 `CAP_SYS_PTRACE` 调试进程等）。 `--cap-drop ALL` 丢弃所有能力，使容器内的进程即使以 root 运行也几乎没有特权操作能力。这是 Docker 安全加固的标准最佳实践。

32.3.3 沙箱路径管理（ `src/agents/sandbox-paths.ts` ）

路径逃逸防护

沙箱环境中最关键的安全问题之一是**路径遍历攻击**——Agent 可能试图通过 `../../etc/passwd` 等路径访问沙箱根目录之外的文件。 `resolveSandboxPath` 函数执行严格的路径验证：

```typescript
// src/agents/sandbox-paths.ts

export function resolveSandboxPath(params: {
  filePath: string;
  cwd: string;
  root: string;
}): { resolved: string; relative: string } {
  const resolved = resolveToCwd(params.filePath, params.cwd);
  const rootResolved = path.resolve(params.root);
  const relative = path.relative(rootResolved, resolved);

  // 检查路径是否逃逸沙箱根目录
  if (relative.startsWith("..") || path.isAbsolute(relative)) {
    throw new Error(`Path escapes sandbox root (${rootResolved}): ${params.filePath}`);
  }

  return { resolved, relative };
}
```

符号链接防护

即使路径在字符串层面没有逃逸，攻击者仍可能通过**符号链接（symlink）**绕过——在沙箱内创建一个指向外部目录的符号链接。 `assertNoSymlink` 函数逐层检查路径中的每个组件：

```typescript
// src/agents/sandbox-paths.ts

async function assertNoSymlink(relative: string, root: string) {
  const parts = relative.split(path.sep).filter(Boolean);
  let current = root;
  for (const part of parts) {
    current = path.join(current, part);
    try {
      const stat = await fs.lstat(current);
      if (stat.isSymbolicLink()) {
        throw new Error(`Symlink not allowed in sandbox path: ${current}`);
      }
    } catch (err) {
      if ((err as { code?: string }).code === "ENOENT") {
        return;  // 路径不存在 → 不构成威胁
      }
      throw err;
    }
  }
}
```

> **衍生解释**：**TOCTOU（Time-of-check to Time-of-use）**是一类常见的安全漏洞——在检查文件属性和实际使用文件之间存在时间窗口，攻击者可以在此期间将正常文件替换为符号链接。OpenClaw 通过使用 `lstat`（不跟随符号链接的 stat）而非 `stat` 来减轻这一风险，但完整的 TOCTOU 防护通常需要操作系统级别的支持（如 Linux 的 `openat` 系统调用配合 `O_NOFOLLOW` 标志）。

Unicode 空格规范化

`expandPath` 函数还处理了一个容易被忽视的攻击向量——Unicode 不间断空格和其他特殊空白字符：

```typescript
const UNICODE_SPACES = /[\u00A0\u2000-\u200A\u202F\u205F\u3000]/g;

function normalizeUnicodeSpaces(str: string): string {
  return str.replace(UNICODE_SPACES, " ");
}
```

这防止攻击者使用特殊空白字符构造看起来合法但实际指向不同位置的路径。

32.3.4 Docker 沙箱容器

容器创建参数

`buildSandboxCreateArgs` 构建 `docker create` 命令的完整参数列表。除了前面提到的安全默认值，还有一个关键的安全标志：

```
// src/agents/sandbox/docker.ts（简化）

export function buildSandboxCreateArgs(params) {
  const args = ["create", "--name", params.name];

  // 标签（用于管理和识别）
  args.push("--label", "openclaw.sandbox=1");
  args.push("--label", `openclaw.sessionKey=${params.scopeKey}`);

  // 安全加固
  if (params.cfg.readOnlyRoot) args.push("--read-only");
  for (const entry of params.cfg.tmpfs) args.push("--tmpfs", entry);
  if (params.cfg.network) args.push("--network", params.cfg.network);
  for (const cap of params.cfg.capDrop) args.push("--cap-drop", cap);
  args.push("--security-opt", "no-new-privileges");  // 关键!

  // 资源限制
  if (params.cfg.pidsLimit > 0) args.push("--pids-limit", String(params.cfg.pidsLimit))
  if (params.cfg.memory) args.push("--memory", params.cfg.memory);
  if (params.cfg.cpus > 0) args.push("--cpus", String(params.cfg.cpus));

  // 可选安全配置
  if (params.cfg.seccompProfile) args.push("--security-opt", `seccomp=${...}`);
  if (params.cfg.apparmorProfile) args.push("--security-opt", `apparmor=${...}`);

  return args;
}
```

> **衍生解释**：`--security-opt no-new-privileges` 是 Linux 内核的 `PR_SET_NO_NEW_PRIVS` 标志。它确保容器内的进程（及其子进程）不能通过 `setuid`、`setgid` 等机制获取新的特权。即使容器内存在一个 setuid-root 的可执行文件，攻击者也无法利用它提升权限。这是 Docker 安全加固中最重要但最容易被遗忘的选项之一。

容器生命周期管理

`ensureSandboxContainer` 管理容器的完整生命周期——创建、复用、配置漂移检测：

```
// src/agents/sandbox/docker.ts（简化）

export async function ensureSandboxContainer(params) {
  const containerName = `${prefix}${slug}`.slice(0, 63);

  // 1. 计算配置哈希
  const expectedHash = computeSandboxConfigHash({ docker, workspaceAccess, workspaceDir

  // 2. 检查容器是否已存在
  const state = await dockerContainerState(containerName);

  if (state.exists) {
    // 3. 配置漂移检测
    const currentHash = await readContainerConfigHash(containerName);
    if (currentHash !== expectedHash) {
      if (isRecentlyUsed) {
        // 正在使用中 → 仅警告，不破坏
        log("Sandbox config changed. Recreate to apply.");
      } else {
        // 空闲 → 自动销毁并重建
        await execDocker(["rm", "-f", containerName]);
        // 后续逻辑将创建新容器
      }
    }
  }

  if (!state.exists) {
    // 4. 创建新容器
    await createSandboxContainer({ name: containerName, cfg, workspaceDir, ... });
  } else if (!state.running) {
    // 5. 容器存在但已停止 → 重启
    await execDocker(["start", containerName]);
  }

  // 6. 更新注册表
  await updateRegistry({ containerName, sessionKey, lastUsedAtMs: Date.now() });
  return containerName;
}
```

配置漂移检测是一个重要的安全特性——当用户修改了沙箱配置（如从 `network: "none"` 改为 `network: "bridge"` ）后，旧容器可能仍在使用旧配置运行。系统通过配置哈希比对来检测这种情况：空闲容器自动重建，活跃容器发出警告。

工作区挂载

容器创建时会将工作区目录挂载到容器中，挂载方式取决于 `workspaceAccess` 配置：

workspaceAccess	挂载方式	说明
`"none"`	沙箱独立目录 → `/workspace`	Agent 无法访问宿主机文件
`"ro"`	宿主机工作区 → `/workspace:ro`	只读访问宿主机文件
`"rw"`	宿主机工作区 → `/workspace`	读写访问宿主机文件

32.3.5 工具白名单 / 黑名单

沙箱工具策略

沙箱环境中不是所有工具都应该可用。 `resolveSandboxToolPolicyForAgent` 解析三级优先级的工具策略：

```
// src/agents/sandbox/tool-policy.ts（简化）

export function resolveSandboxToolPolicyForAgent(cfg?, agentId?): SandboxToolPolicyReso
  // 优先级：Agent 级别 > 全局 > 默认
  const deny  = agentDeny  ?? globalDeny  ?? [...DEFAULT_TOOL_DENY];
  const allow = agentAllow ?? globalAllow ?? [...DEFAULT_TOOL_ALLOW];

  // image 工具默认始终允许（多模态需要）
  if (!deny.includes("image") && !allow.includes("image")) {
    allow.push("image");
  }

  return { allow: expandToolGroups(allow), deny: expandToolGroups(deny), sources };
}
```

默认策略

```ts
// src/agents/sandbox/constants.ts

// 默认允许的工具
const DEFAULT_TOOL_ALLOW = [
  "exec", "process",     // 命令执行
  "read", "write", "edit", "apply_patch",  // 文件操作
  "image",               // 图像处理
  "sessions_list", "sessions_history",     // 会话查看
  "sessions_send", "sessions_spawn",       // 会话操作
  "session_status",      // 会话状态
];

// 默认拒绝的工具
const DEFAULT_TOOL_DENY = [
  "browser",  // 浏览器控制（绕过网络隔离）
  "canvas",   // 画布（可能暴露宿主资源）
  "nodes",    // 节点管理
  "cron",     // 定时任务
  "gateway",  // Gateway 管理
  // 所有渠道 ID（防止沙箱内发送消息）
  ...CHANNEL_IDS,
];
```

通配符匹配

工具名称支持通配符模式匹配：

```ts
// src/agents/sandbox/tool-policy.ts

function compilePattern(pattern: string): CompiledPattern {
  const normalized = pattern.trim().toLowerCase();
  if (normalized === "*") return { kind: "all" };          // 匹配一切
  if (!normalized.includes("*")) return { kind: "exact", value: normalized };
  // 转换为正则表达式
  const escaped = normalized.replace(/[.*+?^${}()|[\]\\]/g, "\\$&");
  return { kind: "regex", value: new RegExp(`^${escaped.replaceAll("\\*", ".*")}$`) };
}

export function isToolAllowed(policy: SandboxToolPolicy, name: string) {
  const normalized = name.trim().toLowerCase();
  // 1. 先检查黑名单 —— 黑名单优先
  if (matchesAny(normalized, deny)) return false;
  // 2. 再检查白名单 —— 空白名单 = 全部允许
  if (allow.length === 0) return true;
  return matchesAny(normalized, allow);
}
```

黑名单优先（deny-first）策略确保了安全性——即使白名单中包含某个工具，如果它同时出现在黑名单中也会被拒绝。

32.3.5 文件系统写入安全加固（v2026.3.9 新增）

v2026.3.9 对沙箱内的文件写入操作进行了全面的安全加固，引入了**固定写入**（Pinned Write）机制和**原子替换**（Atomic Replace）操作。

固定写入助手

`fs-pinned-write-helper.ts`（`src/infra/fs-pinned-write-helper.ts`）通过 Python 子进程执行原子文件操作，防止写入过程中的竞争条件：

```
// src/infra/fs-pinned-write-helper.ts（简化）
async function runPinnedWriteHelper(params: {
  rootPath: string;           // 挂载根目录
  relativeParentPath: string; // 相对父目录
  basename: string;           // 文件名
  mkdir: boolean;             // 是否自动创建目录
  mode: number;               // 文件权限
}) {
  // 1. 使用 O_EXCL 标志独占创建临时文件
  // 2. 写入内容并调用 os.fsync() 确保持久化
  // 3. 使用 os.replace() 原子替换目标文件
  // 4. 返回 dev|ino 标识用于后续校验
}
```

沙箱变异助手

`fs-bridge-mutation-helper.ts` （ `src/agents/sandbox/fs-bridge-mutation-helper.ts` ）构建安全的命令计划，将写入操作封装为可审计的脚本：

```
function buildPinnedWritePlan(params) {
  return {
    checks: [pathSafetyCheck],      // 写入前路径安全检查
    recheckBeforeCommand: true,     // 命令执行前再次检查
    script: ["set -eu", "python3 ..."].join("\n"),
    args: ["write", mountRoot, parentPath, basename, mkdir]
  };
}
```

这套机制的核心安全保证是：临时写入文件**绝不会**在允许的挂载点之外**物化**（Materialize）。即使攻击者在写入过程中操纵了符号链接或替换了父目录， `O_NOFOLLOW` + `O_DIRECTORY` 标志和写入后的 `dev:ino` 校验也能检测到异常。

本节小结

1. **三种沙箱模式**： `off` （关闭）、 `non-main` （推荐，仅隔离外部会话）、 `all` （全部隔离）。
2. **三种作用域**： `session` （最强隔离）、 `agent` （默认，同 Agent 共享）、 `shared` （全局共享）。
3. **安全默认值**：只读根文件系统、无网络、丢弃所有 Linux 能力、禁止特权提升。

4. **路径安全**：路径逃逸检测 + 符号链接检查 + Unicode 空格规范化，防止多种路径遍历攻击。
5. **配置漂移检测**：通过配置哈希比对发现容器与配置不一致，空闲容器自动重建。
6. **工具策略**：Agent 级别 > 全局 > 默认的三级优先级，黑名单优先于白名单。
7. **默认拒绝**：浏览器、渠道发送、Gateway 管理等敏感工具在沙箱中默认被禁用。